

SDP



GitHub Copilot CLI

AI at the Point of Work

Close distance across intent, complexity, context, time, and geography.

"How do I bring AI to where the work actually is — and keep steering it from wherever I am?"

Your day is 70% outside the editor. AI that stays in the IDE misses most of what's actually wrong. **The terminal sees everything. Now AI can too — and follow you home.**

Software Developer

Debug, scaffold, and iterate without leaving the terminal or restarting context

45 min → 8 min

Docker debugging with Plan Mode: 8 attempts down to 2 targeted checks

DevOps Engineer

Triage CI failures, patrol infrastructure, and fix remote servers from any device

25 min → 5 min

CI build failure analysis automated via copilot -p in a GitHub Actions step

CLI Power User

Replace manual trial-and-error with Plan Mode and parallel agent execution

12 GB on the host

Log forensics performed where data lives — no scp, no compliance exposure

PART 1

Plan Mode

Clarify Intent Before Building

Turn ambiguous requests into approved plans before a single line of code is written

PART 2

Operating Modes

Choose Autonomy with Confidence

Interactive, programmatic, fan-out, delegated, and scheduled — with clear permission limits

PART 3

Context Management

Keep the Session Alive

Auto-compaction, repository memory, /rewind recovery, and /chronicle insights

PART 4

Remote Sessions

The Last Distance Falls

Start on any machine, steer from a phone — logs stay where they live

Click any section to jump directly there

Plan Before You Build

Open with the Distance Model, then close the intent gap through a quantified Docker story.



The Distance Model

Five gaps; Plan Mode closes intent first



Docker in 8 Minutes

Questions → plan → 2 checks vs 8 attempts



Rubber Duck Reviewer

Second model reviews plans — enabled by default

Traditional: 8 attempts, 45 minutes, wrong assumptions multiplied
↳ Plan Mode: 2 targeted checks, 8 minutes, root cause found

Five Distances Copilot CLI Removes

Each section of this talk closes a different gap between you and the work

Intent	AI stops guessing, starts asking — Plan Mode clarifies before acting	Plan Mode
Complexity	One task decomposes into parallel subtasks across independent agents	/fleet
Context	Session survives compaction and remembers decisions across sessions	Auto-compact
Time	Work outlives your attention — background delegation frees the terminal	& delegate
Geography	AI meets the problem where it lives — steer from phone or browser	--remote

The session is the unit of work, not the shell

Trial-and-Error vs. Collaborative Planning

Shift+Tab activates Plan Mode — AI asks before acting

✗ Traditional Workflow

- 1 Make request
Describe what you need
- 2 AI assumes intent
Picks one interpretation confidently
- 3 Review output and fix
Wrong assumption discovered too late
- 4 Repeat until correct
8 attempts on average

🔄 45 min, 8 failed attempts

✓ With Plan Mode

- 1 Make request
Describe what you need
- 2 AI asks clarifying questions
Surfaces ambiguities before acting
- 3 Approve the plan
Strategy confirmed before code is written
- 4 Execute with confidence
2 targeted checks — root cause found

✓ 8 min, done

8 debugging attempts → 2 with Plan Mode

Plan Mode in Action: Docker Debugging

Root cause in 8 minutes — not 45

```
$ copilot
> "Debug why the backend container won't start"
🤖 Copilot (Plan Mode):
  Before I start, a few questions:
  1. Check docker-compose config, logs, or both?
  2. Include environment variable analysis?
  3. Any recent changes to services or networking?
> "Start with logs, then config if needed"

🤖 Copilot: Running 2 targeted checks ...
✅ Port 5000 mapped to 5001 in docker-compose.yml – app expects 5000 internally
✅ Fix applied. Container starts successfully.
```

45 min, 8 attempts → 8 min, 2 checks

Choose the Right Mode

Build a decision ladder: interactive diagnosis, headless CI, parallel fan-out, and background delegation.



Interactive vs Headless

copilot for conversation; copilot -p for CI pipelines



/fleet Fan-Out

Decompose a plan into parallel subtasks in one approval



Scheduled Patrols

/every weekday — recurring agent without external cron

Manual CI triage: 25 minutes, context-switching, no pattern history
↳ copilot -p in Actions: 5-minute analysis artifact, zero human intervention

Match the Mode to the Task



Interactive

Unknown root cause, iterative diagnosis, collaborative debugging with context

`copilot`

Plan Mode available

Real-time steering



Programmatic

Headless CI/CD execution, structured output, zero human interaction required

`copilot -p`

Actions integration

`--allow-tool scoped`



Cloud Delegation

20+ min tasks — security audits, refactors. IDE and terminal stay completely free

`& prefix`

PR on completion

GitHub-hosted agent

Build Failure Analysis — Zero Human Intervention

.github/workflows/build.yml

- ```
- name: Analyze failure with Copilot CLI
 if: failure()
 env:
 GITHUB_TOKEN: ${{COPILOT_GITHUB_TOKEN}}
 run: |
 copilot -p 'Analyze the build failure.'
 --allow-tool shell(gh)
 --allow-tool shell(git)
 > failure-analysis.txt

- name: Post analysis as PR comment
 if: failure()
 run: gh pr comment --body-file failure-analysis.txt
```



### 5-Minute Analysis

25-minute manual triage becomes an automated 5-min artifact



### Scoped Permissions

--allow-tool limits the agent to gh and git only



### PR Comment Output

Structured analysis posted directly to the pull request

## /fleet Executes the Plan — Sandboxes Contain the Risk

### ⚡ /fleet Fan-Out

#### Explicit decomposition

Orchestrator assigns subtasks to subagents in parallel

#### Context isolation

Each subagent has its own window — no pollution between tasks

#### Use after Plan Mode approves the strategy

#### Cost note

Each subagent makes independent LLM calls — check multiplier

```
/fleet implement all phases of this auth plan
```

### 🔒 Containment Options

#### Local sandbox (/sandbox enable)

OS containment — no VM, included in Copilot seat

#### Cloud sandbox (--cloud)

Isolated GitHub-hosted environment — metered

#### disableBypassPermissionsMode

Enterprise policy — prevents --yolo in org environments

**Never use --yolo on a remote production session**

## The CLI as a Recurring Agent Runner

Schedule patrol prompts in plain language — no external cron required (experimental)

```
$ copilot --experimental
> "/every weekday at 9am"
 Summarize overnight PRs and post to Slack #dev-standup
> "/every 30min"
 Check pod health in staging.
 Alert via remote session if anything is unhealthy.
```

---

- ✓ Session-scoped – restarts with `--resume`; fires while session is running
- ✓ Pairs with `--remote`: patrol runs on the server, you steer from anywhere

*Requires /experimental on · minimum 10s · maximum 1 day*

# Context That Carries

Two ideas: the durable session and turn-level recovery — the foundation that makes `--remote` worth using.



## Infinite Sessions

Auto-compaction at 95% — sessions worth reconnecting to



## `/rewind` Recovery

Roll back a bad turn — Copilot edits undone, your edits kept



## `/chronicle` Insights

Sessions → standup, tips, better `copilot-instructions.md`

Context lost every 12–20 turns; restart and re-explain every session  
↳ `/resume` from any machine – session, memory, and repo context travel with you

## The Session Is the Unit of Work — Not the Shell

### ∞ Auto-Compaction

#### Fires at 95% token limit

Compresses history in background — no interruptions

Important decisions and facts persist; verbose output pruned

#### /resume from any machine

Session + repo memory + context travel together

```
/context # token usage breakdown
/usage # session stats: duration, edits,
/compact # compress manually anytime
```

### 🔄 /rewind Recovery

#### Roll back to an earlier turn

No Git required — works on any folder

#### Conversation only

Rewind the chat, leave files as they are

#### Conversation + files

Restore Copilot-changed files; skip files you edited

Tracked across editing tools, shell commands, and sub-agents

```
> /rewind
Pick turn before the bad refactor
Choose: Conversation + files
```

## /chronicle Turns Session History into Actionable Intelligence



### /chronicle standup

Generates a standup report from recent sessions — what you built, what you fixed



### /chronicle tips

Reviews your usage patterns and surfaces personalized improvement suggestions



### /chronicle improve

Analyzes repo sessions and suggests additions to `.github/copilot-instructions.md`



### Open-ended queries

Ask what you worked on, what you repeated, what you could automate

# Work from Anywhere

Geography was the last constraint. `--remote` removes it — AI goes to the machine, you steer from anywhere.



## Steer from a Phone

Start on the server, approve from GitHub Mobile mid-walk



## Data Stays Local

12 GB logs analyzed on the host — no scp, no compliance risk



## Composable

Plan Mode + `/fleet` + Memory all work inside `--remote`

```
SSH in, copy logs, paste into chat, lose context switching tools
↳ copilot --remote on the server — steer the live session from your phone
```

## Start on the Server — Steer from Anywhere

prod-server-3 (us-east-1)

```
SSH into the server
ssh ops@prod-server-3.us-east-1
```

```
Start a remote session
$ copilot --remote
```

```
🔑 Remote session started.
Monitor and steer from:
 https://github.com/copilot/sessions/abc123
```

```
Or enable mid-session:
> /remote
```

### 👁 See in Real-Time

Watch Copilot work from any browser or GitHub Mobile

### 👋 Approve from Anywhere

Grant or deny tool permissions from your phone mid-walk

### 🔄 Steer and Resume

Inject prompts or /resume the session from a new machine

## Production Incident — Walking to a Meeting

The session lives on the server. You steer from your phone.

### ✗ Without --remote

- 1 Alert fires at 2 PM  
You're walking to a 2:05 meeting
- 2 Wait until after meeting  
30+ min of unresolved degradation
- 3 SSH in from laptop  
Re-establish context from scratch
- 4 Manually scp 12 GB logs  
Compliance and bandwidth risk

⌚ 30+ min delay, compliance exposure

### ✓ With --remote

- 1 Alert fires at 2 PM  
You're walking to a 2:05 meeting
- 2 Open GitHub Mobile  
Live session URL — copilot is already on the server
- 3 Approve tool calls from phone  
Copilot runs /fleet log analysis
- 4 12 GB stays on the host  
Analysis runs where the data lives

✓ Diagnosed in-meeting, data never moved

--remote: execution locality + operator locality, no data transfer

## --remote Composes with Every Capability



### Plan Mode + --remote

AI asks clarifying questions before touching a production server — safer phone approvals



### /fleet + --remote

Parallel subtasks across multiple environments from one approved plan on one host



### Scheduled + --remote

Infrastructure patrol on the server — fires on schedule, you review from any device



### Memory + --remote

/resume from a new device — repository memory and session context travel with you

## From Trial-and-Error Commands to AI Sessions That Follow the Work

### Before

Re-explain codebase conventions every session restart

SSH in, manually copy logs, paste into chat, lose context switching tools

One task at a time — terminal blocked during long-running agent work

Skip planning under deadline pressure and spiral into 8 failed attempts

### After

Repository memory carries conventions into every future session automatically

--remote puts AI on the server — steer log forensics from any device

& delegates 20+ min tasks to GitHub's agent — IDE and terminal stay completely free

Plan Mode turns 45 min of trial-and-error into an 8-min approved strategy

**8 min**

Docker debugging with Plan Mode (was 45 min, 8 attempts)

**5 min**

CI build failure analysis via copilot -p (was 25 min manual)

**0 scp**

12 GB log forensics performed on the host — data never moves

## ✓ What You Can Do Today

### Today

- ❑ Install Copilot CLI: `npm install -g @github/copilot`
- ❑ Press Shift+Tab before your next debugging session — activate Plan Mode
- ❑ Run `/rewind` after any wrong-turn refactor — no Git required

### This Week

- ❑ Add a `copilot -p` step to CI for automated build failure analysis
- ❑ Start a `--remote` session on a staging server and steer from GitHub Mobile
- ❑ Run `/chronicle improve` to improve your `.github/copilot-instructions.md`

### This Month

- ❑ Replace log-copy incident workflows with `--remote` sessions on production hosts
- ❑ Use `/fleet` after Plan Mode to parallelize your largest recurring refactors
- ❑ Enable `/experimental` and set up a `/every weekday` patrol for overnight PRs

### 🔑 Key Takeaway

The session is the unit of work — start on any machine, finish from wherever you are.

 Core Documentation[About GitHub Copilot CLI](#)

Core concepts, session model, and operating modes

[Install Copilot CLI](#)

Setup instructions for all platforms

[CLI Command Reference](#)

Full command syntax including `/rewind`, `/fleet`, `/chronicle`

[Steering a session remotely](#)

Remote session setup and mobile steering

 Related Talks[Copilot Memory](#)

Repository memory and cross-session context in depth

[Agent Plugins 1.0](#)

Portable skills, MCP config, and custom agents for the CLI

[CLI + Azure MCP](#) How-To Guides[Scheduling prompts](#)

`/every` and `/after` — recurring agent workflows

[Running tasks in parallel with `/fleet`](#)

Fan-out orchestration and subagent management

[Cloud and local sandboxes](#)

`/sandbox enable` and `--cloud` for containment

[Using `/chronicle`](#)

Standup, tips, and improve `copilot-instructions.md`



# GitHub Copilot CLI

AI at the Point of Work — Close distance across intent, complexity, context, time, and geography

**8 min**

Docker root cause — Plan Mode, 2 checks  
vs 8 attempts

**5 min**

CI analysis — copilot -p automates a 25-  
min triage step

**12 GB**

Log forensics on the host — data stays, --  
remote travels

What's your longest recurring debugging or triage loop — and which mode would you reach for first?

## Interactive Mode — Agent Environment

|                            |                                                |
|----------------------------|------------------------------------------------|
| <code>/init</code>         | Initialize Copilot instructions for this repo  |
| <code>/agent</code>        | Browse and select from available agents        |
| <code>/skills</code>       | Manage skills for enhanced capabilities        |
| <code>/mcp</code>          | Manage MCP server configuration                |
| <code>/plugin</code>       | Manage plugins and plugin marketplaces         |
| <code>/instructions</code> | View & toggle custom instruction files         |
| <code>/env</code>          | Show loaded instructions, MCPs, skills, agents |

### What this gives you

Everything that shapes the agent's behavior in this session — instructions, custom agents, skills, MCP servers, and plugins.

### Start here

`/init` once per repo, then `/env` any time you wonder "what does the agent actually know right now?"

## Interactive Mode — Models & Subagents

|                         |                                        |
|-------------------------|----------------------------------------|
| <code>/model</code>     | Select AI model to use                 |
| <code>/autopilot</code> | Toggle autopilot mode                  |
| <code>/fleet</code>     | Parallel subagent execution            |
| <code>/delegate</code>  | Send session to GitHub → create a PR   |
| <code>/tasks</code>     | View & manage tasks (subagents, shell) |
| <code>/sidekicks</code> | View running sidekick agents           |

### Three execution gears

- Interactive — you approve each step
- Autopilot — agent runs until done
- Fleet — multiple subagents in parallel

### Hand-off vs parallelize

`/delegate` ships the work to GitHub coding agent;  
`/fleet` stays local and fans out subagents.

## Interactive Mode — Code

|                              |                                          |
|------------------------------|------------------------------------------|
| <code>/plan</code>           | Implementation plan before coding        |
| <code>/diff</code>           | Review the changes made in current di    |
| <code>/pr</code>             | Operate on pull requests for this branch |
| <code>/review</code>         | Run code review agent on changes         |
| <code>/ide</code>            | Connect to an IDE workspace              |
| <code>/lsp</code>            | Manage language server configuration     |
| <code>/terminal-setup</code> | Configure shift+enter for multiline      |

### Plan → Code → Review

`/plan` drafts the approach, the agent codes, then `/diff` and `/review` let you inspect & refine before opening a PR.

### IDE bridge

`/ide` pairs the CLI with an open editor — file edits open as native diffs and selections flow back to Copilot.

## Interactive Mode — Session

|                       |                                         |
|-----------------------|-----------------------------------------|
| <code>/resume</code>  | Switch sessions (id / task id / name)   |
| <code>/session</code> | View & manage sessions                  |
| <code>/rename</code>  | Rename or auto-name current session     |
| <code>/context</code> | Show context window token usage         |
| <code>/usage</code>   | Session usage metrics & stats           |
| <code>/compact</code> | Summarize history to free context       |
| <code>/memory</code>  | Toggle cross-session fact recall        |
| <code>/rewind</code>  | Undo last turn + revert file changes    |
| <code>/share</code>   | Export to markdown / HTML / gist        |
| <code>/copy</code>    | Copy last response to clipboard         |
| <code>/remote</code>  | Toggle control from GitHub web / mobile |

### Context hygiene

Watch `/context` . When it's > 70% full, run `/compact` with focus instructions instead of starting over.

### Sessions are persistent

All sessions live in `~/.copilot/session-store.db` . `/resume` picks any one up — even days later, by name or ID prefix.

## Interactive Mode — Permissions

|                                   |                                             |
|-----------------------------------|---------------------------------------------|
| <code>/allow-all</code>           | Enable all permissions (tools, paths, URLs) |
| <code>/add-dir</code>             | Whitelist a directory for file access       |
| <code>/list-dirs</code>           | Show all allowed directories                |
| <code>/cwd</code>                 | Change or show working directory            |
| <code>/reset-allowed-tools</code> | Clear the list of allowed tools             |

### Three permission surfaces

- Tools — what the agent can invoke
- Paths — what files it can touch
- URLs — what domains it can reach

### Denial always wins

Deny rules override `/allow-all`. Safe pattern: allow broadly, deny specifically (e.g. `shell(git push)`).

## Interactive Mode — Help & More

|                             |                                         |
|-----------------------------|-----------------------------------------|
| <code>/research</code>      | Deep research via GitHub + web          |
| <code>/chronicle</code>     | Session history insights                |
| <code>/changelog</code>     | CLI version history (+ AI summary)      |
| <code>/new</code>           | Start a new conversation                |
| <code>/clear</code>         | Abandon session and start fresh         |
| <code>/keep-alive</code>    | Prevent system sleep during work        |
| <code>/voice</code>         | Dictation via Foundry Local             |
| <code>/theme</code>         | View or set color mode                  |
| <code>/streamer-mode</code> | Hide model names / quota (screen share) |
| <code>/experimental</code>  | Enable / disable experimental features  |
| <code>/feedback</code>      | Provide feedback about the CLI          |
| <code>/help</code>          | Show help for interactive commands      |

### Long-running work

`/research` spins up a deep-dive task; pair with `/keep-alive` so your laptop doesn't sleep mid-run.

### Demo & screen-share kit

`/streamer-mode` hides quota and preview model names; `/theme` swaps light/dark to match your slides.

## Non-Interactive Mode — Scriptable AI

# One-shot prompt, exits after response

```
copilot -p "Fix the failing test in auth.test.ts" --allow-all
```

# Quiet output – only agent response, no stats (great for piping)

```
copilot -p "Summarize this PR" --silent | pbcopy
```

# Autopilot – continues autonomously until done

```
copilot -p "Migrate all API routes to use the new auth middleware" --allow-all --autopilot
```

# Share session output as a GitHub gist

```
copilot -p "Audit this repo for security issues" --allow-all --share-gist
```

# Structured JSON output for CI pipelines

```
copilot -p "Run tests and report failures" --allow-all --output-format json
```

## Permissions — Granular Control

```
Allow specific tool categories
copilot --allow-tool=write --allow-tool=shell

Allow all git except push
copilot --allow-tool='shell(git:*)' \
--deny-tool='shell(git push)'

Restrict to specific directories
copilot --add-dir ~/workspace --add-dir /tmp

Allow GitHub API access
copilot --allow-url=github.com

Enable everything – tools, paths, URLs
copilot --yolo

Same as:
copilot --allow-all-tools \
--allow-all-paths \
--allow-all-urls

--allow-all shorthand for all three
--yolo alias for --allow-all

Required for non-interactive (-p) mode
copilot -p "... " --allow-all-tools
```

## `copilot init` — Bootstrap Your Repo

*Analyzing codebase structure ...*

*Detecting build system: npm*

*Detecting test framework: jest*

*Detecting language: TypeScript*

✓ Created `.github/copilot-instructions.md`

### What it generates

Analyzes your repo with read-only tools and writes `.github/copilot-instructions.md` with:

- Build & test commands
- Coding conventions
- Project structure overview
- Technology stack details

### Why it matters

Every future session in this repo automatically loads these instructions — no need to re-explain your project to Copilot.

## `copilot login` — Authentication

```
Interactive browser flow (default)
copilot login

GitHub Enterprise Cloud (data residency)
copilot login --host https://example.ghe.com

Headless / CI - use env var instead of login
COPILOT_GITHUB_TOKEN=github_pat_... copilot

Token precedence order:
COPILOT_GITHUB_TOKEN
GH_TOKEN
GITHUB_TOKEN
```

### Supported token types

- Fine-grained PATs (v2) with *Copilot Requests* permission
- OAuth tokens from GitHub Copilot CLI app
- OAuth tokens from GitHub CLI (gh) app

✗ Classic PATs (ghp\_...) are not supported

### Token storage

Stored in system credential store. Falls back to plain text in `~/.copilot/` if no credential store is available.

## `copilot plugin` — Extend with Plugins

```
Browse the default marketplace
copilot plugin marketplace browse copilot-plugin

Install from GitHub repository
copilot plugin install github/my-plugin

Install from local directory
copilot plugin install ./my-local-plugin

List installed plugins
copilot plugin list

Update / uninstall
copilot plugin update my-plugin
copilot plugin uninstall my-plugin
```

### What plugins can add

- Additional skills & agents
- Git hooks
- MCP servers
- LSP servers

### Built-in marketplaces

```
copilot-plugins github/copilot-plugins
awesome-copilot github/awesome-copilot
```

## copilot help config — Key Settings

### Behavior

**model** – AI model (gpt-5.4, claude-sonnet-4.6, ... )  
**autoUpdate** – auto-download CLI updates (default: true)  
**memory** – agentic cross-session fact recall (default: true)  
**keepAlive** – prevent system sleep (off / on / busy)  
**experimental** – enable experimental features  
**renderMarkdown** – render markdown in terminal  
**respectGitignore** – hide gitignored files from @ picker  
**includeCoAuthoredBy** – add Co-authored-by to commits  
**compactPaste** – collapse large pastes (>10 lines)  
**streamerMode** – hide model names & quota (for screen sharing)

### Terminal

**mouse** – enable mouse support in alt screen  
**beep** – beep when attention required

### Security

**allowedUrls** – pre-approved URL/domain list (supports \*.domain)  
**deniedUrls** – blocked domains (takes precedence)  
**trustedFolders** – folders with persisted read/exec approval

### Available models

claude-sonnet-4.6    claude-sonnet-4.5    claude-haiku-4.5  
claude-opus-4.8    claude-opus-4.7    claude-opus-4.6  
claude-opus-4.6-fast    claude-opus-4.5  
gpt-5.5    gpt-5.4    gpt-5.2  
gpt-5.3-codex    gpt-5.2-codex  
gpt-5.4-mini    gpt-5-mini

### Teams

**companyAnnouncements** – messages shown in banner on startup

## copilot help environment — Key Env Vars

### Authentication

`COPILOT_GITHUB_TOKEN` – PAT / OAuth token (highest precedence)

`GH_TOKEN` – GitHub CLI token (second)

`GITHUB_TOKEN` – CI token (third)

`GH_HOST` – GitHub Enterprise hostname

`COPILOT_GH_HOST` – Copilot-only host (overrides `GH_HOST`)

### Behavior

`COPILOT_ALLOW_ALL=true` – skip all confirmations

`COPILOT_MODEL` – override AI model

`COPILOT_AUTO_UPDATE=false` – disable auto-update

`COPILOT_HOME` – override `~/.copilot` directory

`COPILOT_OFFLINE=true` – no network (needs BYOK)

`COPILOT_CUSTOM_INSTRUCTIONS_DIRS` – extra `AGENTS.md` dirs

### BYOK – Bring Your Own Key

`COPILOT_PROVIDER_BASE_URL` – custom API endpoint (activates BYOK)

`COPILOT_PROVIDER_TYPE` – openai / azure / anthropic

`COPILOT_PROVIDER_API_KEY` – API key

`COPILOT_PROVIDER_BEARER_TOKEN` – bearer token (takes precedence over API key)

`COPILOT_PROVIDER_WIRE_API` – completions / responses (use responses for GPT-5)

`COPILOT_PROVIDER_AZURE_API_VERSION` – Azure API version

### Proxy & Output

`HTTP_PROXY` / `HTTPS_PROXY` – outbound proxy URL

`NO_PROXY` – bypass list (comma-separated)

`NO_COLOR` – disable color output

`PLAIN_DIFF=true` – disable rich diff

## BYOK — Bring Your Own Key

```
Ollama (local, no auth required)
COPILOT_PROVIDER_BASE_URL=http://localhost:11434/v1 \
COPILOT_MODEL=deepseek-coder-v2:16b \
copilot

Custom OpenAI-compatible endpoint
COPILOT_PROVIDER_BASE_URL=https://my-api.example.com/v1 \
COPILOT_PROVIDER_API_KEY=sk- ... \
COPILOT_MODEL=gpt-4 \
copilot

Azure OpenAI with deployment name
COPILOT_PROVIDER_TYPE=azure \
COPILOT_PROVIDER_BASE_URL=https://my.openai.azure.com \
COPILOT_PROVIDER_MODEL_ID=gpt-4 \
COPILOT_PROVIDER_WIRE_MODEL=my-gpt4-deployment \
copilot
```

### Provider types

**openai** – default; covers Ollama, vLLM, Foundry Local  
**azure** – Azure OpenAI (host URL only)  
**anthropic** – Anthropic API

### When to use

- Air-gapped or offline environments
- Enterprise contracts with specific model providers
- Local development with Ollama / Foundry Local
- Cost control with self-hosted models

## `/fleet` — Parallel Subagent Execution

*Refactor auth module, add unit tests,  
update API docs, run the linter.*

`/fleet`

✓ Plan — 4 subtasks identified

[subagent-1] Refactor auth module

[subagent-2] Write unit tests

[subagent-3] Update API documentation

[subagent-4] Run linter across repo

*Running in parallel...*

✓ [subagent-4] Linter: 3 warnings fixed

✓ [subagent-1] Auth refactored

✓ [subagent-3] 12 endpoints updated

✓ [subagent-2] 24 tests added, passing

✓ All subtasks complete

### How it works

- Main agent acts as orchestrator
- Decomposes prompt into independent subtasks
- Runs in parallel, each with its own context
- Manages dependencies; merges results

### Best used for

- Multi-file refactors + test generation
- Audits + fixes across modules
- Large dependency updates in monorepos

### Power combos

Shift+Tab → plan → Accept + autopilot + `/fleet`

*"Use GPT-5.3-Codex to refactor..."*

*"Use @test-writer to create tests..."*

## Where the Magic Lives — ~/.copilot/

How Copilot remembers everything — in-session and between sessions

|                        |                        |
|------------------------|------------------------|
| agents/                | custom agents          |
| skills/                | skill packs            |
| ide/                   | IDE bridge             |
| logs/ session-state/   | in-session state       |
| pkg/                   | CLI & auto-update      |
| restart/               | restart staging        |
| config.json            | settings               |
| permissions-*.json     | allow/deny rules       |
| session-store.db       | past sessions (SQLite) |
| command-history-*.json | prompt history         |

### In-session

- `session-state/` — context, tasks, subagent state
- `ide/` — live editor bridge
- `logs/` — execution logs

### Between-session

- `session-store.db` — all conversations, resumable
- `command-history-*.json` — up-arrow recall
- `permissions-*.json` — persistent approvals

### Portable capabilities

- `agents/` & `skills/` follow you everywhere
- Override with `$COPILOT_HOME`